

Computer Networks Lab

Fall 2025

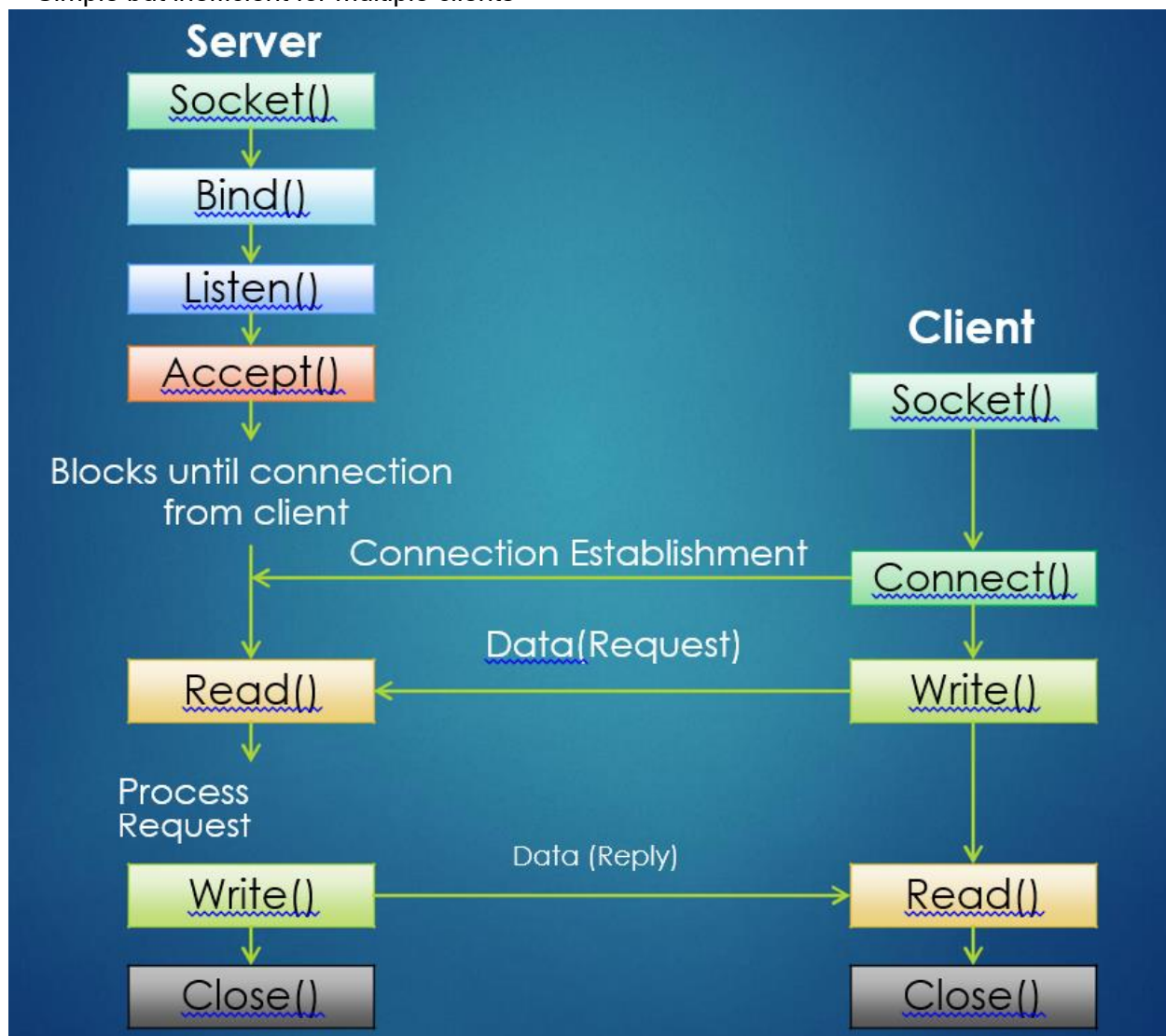
Week 7

Socket Programming: Concurrent Servers

1. Introduction to Concurrent Servers

By now you should be familiar with basic TCP and UDP servers and clients. In previous labs, we used the **iterative server model** where:

- Server serves one client at a time
- Other clients must wait in queue
- Simple but inefficient for multiple clients



The Problem with Iterative Servers

Client 1 → Server (serving...) → Client 2 (waiting...) → Client 3 (waiting...)

Issues:

- Long waiting times for clients
- Server idle while waiting for one client
- Poor scalability for real-world applications
- Bottleneck in high-traffic scenarios

The Solution: Concurrent Servers

Concurrent servers can handle multiple clients simultaneously by creating separate execution contexts (processes or threads) for each client.

2. Concurrency Methods Comparison

Feature	fork() (Process)	pthread (Thread)
Memory	Separate memory space	Shared memory
Resource Usage	Heavy (full process copy)	Lightweight
Communication	Complex (IPC needed)	Simple (shared variables)
Safety	High (isolated)	Requires synchronization
Speed	Slower creation	Faster creation
Best For	Independent tasks	Shared data tasks

3. Method 1: Using fork() - Process-Based Concurrency

What is fork()?

fork() is a system call that creates a new process by duplicating the calling process.

`pid_t fork(void);`

Returns:

- **0** → In child process
- **Positive PID** → In parent process (child's process ID)
- **-1** → Error

How fork() Works

The diagram illustrates the `fork()` system call and its associated code and annotations. It shows a C code snippet for creating a child process and managing its execution.

```

int pid;
int status = 0;

pid = fork();
if (pid > 0) {
    /* parent */
    ....
    pid = wait(&status);
} else {
    /* child */
    ....
    exit(status);
}

```

Annotations explaining the code:

- The `fork` syscall returns a zero to the child and the child process ID to the parent** (points to the `pid = fork();` line).
- Parent uses `wait` to sleep until child returns and** (points to the `pid = wait(&status);` line).
- Fork creates an exact copy of the parent process** (points to the `fork()` function).
- Wait variants allow wait on a specific child or** (points to the `wait(&status);` line).
- Child process passes status back to parent on `exit`, to report success/failure** (points to the `exit(status);` line).

Key Points:

- Child gets copy of parent's memory
- Both processes run independently
- Changes in one don't affect the other
- Child inherits open file descriptors (like sockets)

Concurrent Server Logic with fork()

Main Server Process (Parent):

1. Create socket
2. Bind to port
3. Listen for connections
4. while(1):
 - a. Accept client connection
 - b. fork() - Create child process
 - c. In child: Handle client communication
 - d. In parent: Close client socket, continue accepting

4. Method 1 Server Code Explanation**New Headers for fork():**

```
#include <sys/types.h>    // For pid_t type
#include <unistd.h>        // For fork(), close()
```

setsockopt() - Socket Options

This is a system call that allows you to configure how a socket behaves at the operating system level.

Its general form is this:

*int setsockopt(int sockfd, int level, int option_name, const void *option_value, socklen_t option_len);*

Purpose: Allows immediate port reuse after server restart. Used to set special options on sockets.

Example: Immediate port reuse after server restart (avoiding "Address already in use" error).

Parameters:

1. **sockfd** → The socket file descriptor
 - The socket you created using socket().
2. **level** → Protocol level at which the option is applied
 - **SOL_SOCKET** → Apply option at **socket level**.
 - **Other examples:** IPPROTO_TCP (for TCP-specific options), IPPROTO_IP (for IP level).
3. **option_name** → The specific option you want to set from the level you want to set
 - Common one for SOL_SOCKET: SO_REUSEADDR
 - Allows reusing a port immediately after program crash/restart.
 - Without this: you may get "**Address already in use**" error when rebinding.
4. **option_value** → Pointer to the value you want to set
 - Usually an integer (e.g., int opt = 1;)
 - &opt is passed to setsockopt()
5. **option_len** → Size of the option value
 - Example: sizeof(opt)

```
int opt = 1;
```

```
// Allow address reuse
if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}
```

Main Accept Loop

```
while (1) {
    clientSocket = accept(sockfd, (struct sockaddr*)&cliAddr, &addr_size);

    printf("Connection from %s:%d\n",
        inet_ntoa(cliAddr.sin_addr),    // IP address
        ntohs(cliAddr.sin_port));      // Port number

    if ((childpid = fork()) == 0) {
        // CHILD PROCESS CODE
        close(sockfd); // Don't need listening socket

        // Communication loop with client
        while (1) {
            recv(clientSocket, buffer, BUFFER_SIZE, 0);
            // Process and respond
            send(clientSocket, buffer, strlen(buffer), 0);
        }

        close(clientSocket);
        exit(0); // Child terminates
    }

    // PARENT PROCESS continues here
    close(clientSocket); // Parent doesn't need this
}
```

Important fork() Concepts

Why Parent Closes clientSocket?

Parent doesn't communicate with client - child handles it. Closing prevents resource leak.

Why close() in different processes?

```
// After fork():
if (childpid == 0) {
    // CHILD PROCESS
    close(sockfd); // Close listening socket (not needed)
    // Handle client...
    close(clientSocket); // Close client socket
    exit(0); // Child exits
}
```

```
// PARENT PROCESS
close(clientSocket);    // Close client socket (child handles it)
// Continue accepting new clients
```

Reason: Each socket has a reference count. Must close in both processes to fully release resources.

5. Method 2: Using Threads - Thread-Based Concurrency

Multithreading is a programming technique where a single process is divided into multiple lightweight execution units called **threads**.

Key Features of Threads

- Each thread runs **independently**, but all threads **share the same memory space** and resources of the parent process.
- Thread creation and context switching are **faster and more efficient** than creating processes with `fork()`.
- Since threads share memory, **communication is easier** (no need for inter-process communication mechanisms).
- However, shared memory requires **synchronization tools** (locks, semaphores, mutexes) to prevent **race conditions** and **data corruption**.

Use in Concurrent Servers

- In concurrent servers, **multithreading** allows handling **multiple clients simultaneously**.
- The **main thread** listens for incoming client connections.
- For each new client, the server **creates a worker thread** to manage communication with that client.
- This way, multiple clients can be served **in parallel**.

6. Method 2 Server Code Explanation

pthread Library

```
#include <pthread.h>    // for threads
```

Compile with: `gcc server.c -o server -lpthread`

Main Functions:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

```
int pthread_detach(pthread_t thread);
```

Thread Function Signature

```
void *handle_client(void *arg) {
    // Thread code here
    return NULL; // Must return void*
}
```

Parameters:

- **void *arg** → Can pass any data (cast to appropriate type)

Return:

- **void *** → Can return any pointer (usually NULL)

Creating Thread for Each Client

```
// Allocate memory for client socket (thread will free it)
int *pclient = malloc(sizeof(int));
*pclient = client_fd;

// Create thread
pthread_create(&tid, NULL, handle_client, pclient);

// Detach thread (automatic resource cleanup)
pthread_detach(tid);
```

Why malloc() for client_fd?

Problem: If we pass &client_fd directly, it's shared across all threads.

Solution: Allocate separate memory for each thread.

// WRONG - All threads share same variable:

```
pthread_create(&tid, NULL, handle_client, &client_fd);
```

// CORRECT - Each thread gets own copy:

```
int *pclient = malloc(sizeof(int));
*pclient = client_fd;
pthread_create(&tid, NULL, handle_client, pclient);
```

Thread Function Implementation

```
void *handle_client(void *arg) {
    // Extract client socket from argument
    int client_fd = *((int *)arg);

    // Free the allocated memory
    free(arg);

    char buffer[BUFFER_SIZE];

    // Communication loop
    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytes = recv(client_fd, buffer, BUFFER_SIZE, 0);

        if (bytes <= 0) {
            printf("Client disconnected.\n");
            close(client_fd);
            break;
        }
    }
}
```

```

        printf("Client: %s\n", buffer);
        send(client_fd, buffer, strlen(buffer), 0);
    }

    return NULL; // Thread terminates
}

```

Purpose of `pthread_detach()`

```
pthread_detach(tid);
```

Without detach: Must call `pthread_join()` to clean up resources.

With detach: Resources automatically freed when thread terminates.

7. Complete Code (Method 1: Using the `fork()` call)

Server Side code:

```

#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h> // For pid_t type
#include <unistd.h>    // For fork(), close()

#define PORT 4444
#define BUFFER_SIZE 1024

int main() {
    int sockfd, ret;
    struct sockaddr_in serverAddr;
    int clientSocket;
    struct sockaddr_in cliAddr;
    socklen_t addr_size;
    pid_t childpid;
    int opt = 1;
    char buffer[BUFFER_SIZE];

    // Creates a TCP socket id from IPV4 family
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        printf("Error in connection.\n");
        exit(1);
    }
}

```

```

printf("Server Socket is created.\n");

// Allow address reuse
if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}

// Initializing address structure with NULL
memset(&serverAddr, '\0', sizeof(serverAddr));
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

// Binding the socket id with the socket structure
ret = bind(sockfd, (struct sockaddr*)&serverAddr, sizeof(serverAddr));
if (ret < 0) {
    printf("Error in binding.\n");
    exit(1);
}

if (listen(sockfd, 10) == 0) {
    printf("Listening...\n\n");
}

addr_size = sizeof(cliAddr);

while (1) {
    // Accept client connection
    clientSocket = accept(sockfd, (struct sockaddr*)&cliAddr, &addr_size);
    if (clientSocket < 0) {
        exit(1);
    }

    // Displaying information of connected client
    printf("Connection accepted from %s:%d\n", inet_ntoa(cliAddr.sin_addr),
ntohs(cliAddr.sin_port));

    // Create a child process for the connected client
    if ((childpid = fork()) == 0) {
        // Close the listening socket in the child process
        close(sockfd);
    }
}

```



```

while (1) {
    // Clear the buffer
    memset(buffer, 0, BUFFER_SIZE);

    // Receive message from the client
    int recv_bytes = recv(clientSocket, buffer, BUFFER_SIZE, 0);
    if (recv_bytes <= 0) {
        printf("Client disconnected.\n");
        break;
    }

    // Print the client's message
    printf("Client: %s\n", buffer);

    // Respond back to the client
    send(clientSocket, buffer, strlen(buffer), 0);
}

// Close the client socket and exit child process
close(clientSocket);
exit(0);
}

// Close the client socket in the parent process
close(clientSocket);
}

// Close the listening socket
close(sockfd);
return 0;
}

```

Client side code:

```

#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
#define PORT 4444
#define BUFFER_SIZE 1024

```

```
int main() {
    int clientSocket, ret;
    struct sockaddr_in serverAddr;
    char buffer[BUFFER_SIZE];
    // Creating socket id
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (clientSocket < 0) {
        printf("Error in connection.\n");
        exit(1);
    }
    printf("Client Socket is created.\n");

    // Initializing server address structure
    memset(&serverAddr, '\0', sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Connect to the server
    ret = connect(clientSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr));
    if (ret < 0) {
        printf("Error in connection.\n");
        exit(1);
    }
    printf("Connected to Server.\n");

    while (1) {
        // Clear the buffer
        memset(buffer, 0, BUFFER_SIZE);

        // Input message to send to the server
        printf("Enter message: ");
        fgets(buffer, BUFFER_SIZE, stdin);

        // Send message to the server
        send(clientSocket, buffer, strlen(buffer), 0);

        // Clear the buffer to receive the server's response
        memset(buffer, 0, BUFFER_SIZE);

        // Receive the response from the server
        if (recv(clientSocket, buffer, BUFFER_SIZE, 0) < 0) {
            printf("Error in receiving data.\n");
        } else {
```

```

        printf("Server: %s\n", buffer);
    }
}

// Close the client socket
close(clientSocket);
return 0;
}

```

8. Complete Code (Method 2: Using the pthread_create() call)

Server side code:

```

#include <arpa/inet.h>
#include <netinet/in.h>
#include <pthread.h>    // for threads
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 7000
#define BUFFER_SIZE 1024

// Function prototype for the thread function
void *handle_client(void *arg);

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;
    pthread_t tid;

    // Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    printf("Server socket created.\n");

    // Configure server address
    server_addr.sin_family = AF_INET;

```

```

server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
memset(server_addr.sin_zero, '\0', sizeof(server_addr.sin_zero));

// Bind
if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Bind failed");
    exit(EXIT_FAILURE);
}
printf("Bind to port %d successful.\n", PORT);

// Listen
if (listen(server_fd, 10) == 0) {
    printf("Listening...\n");
} else {
    perror("Listen failed");
    exit(EXIT_FAILURE);
}

// Accept connections in a loop
while (1) {
    addr_size = sizeof(client_addr);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &addr_size);
    if (client_fd < 0) {
        perror("Accept failed");
        continue;
    }

    printf("Connection accepted from %s:%d\n",
           inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

    // Create a thread for each client
    int *pclient = malloc(sizeof(int));
    *pclient = client_fd;
    pthread_create(&tid, NULL, handle_client, pclient);

    // Detach thread so its resources are freed automatically
    pthread_detach(tid);
}

close(server_fd);
return 0;
}

```

```
// Thread function to handle each client
void *handle_client(void *arg) {
    int client_fd = *((int *)arg);
    free(arg);

    char buffer[BUFFER_SIZE];
    int bytes_received;

    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        bytes_received = recv(client_fd, buffer, BUFFER_SIZE, 0);

        if (bytes_received <= 0) {
            printf("Client disconnected.\n");
            close(client_fd);
            break;
        }

        printf("Client says: %s\n", buffer);
        send(client_fd, buffer, strlen(buffer), 0); // Echo back
    }

    return NULL;
}
```

Client side code:

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 7000
#define BUFFER_SIZE 1024

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE];

    // Create socket
```

```
client_fd = socket(AF_INET, SOCK_STREAM, 0);
if (client_fd < 0) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

// Connect to server
if (connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Connection failed");
    exit(EXIT_FAILURE);
}
printf("Connected to server.\n");

while (1) {
    printf("Enter message: ");
    fgets(buffer, BUFFER_SIZE, stdin);
    buffer[strcspn(buffer, "\n")] = '\0'; // Remove newline

    if (strcmp(buffer, "exit") == 0) {
        send(client_fd, buffer, strlen(buffer), 0);
        break;
    }

    send(client_fd, buffer, strlen(buffer), 0);

    memset(buffer, 0, BUFFER_SIZE);
    int bytes_received = recv(client_fd, buffer, BUFFER_SIZE, 0);
    if (bytes_received > 0) {
        printf("Server: %s\n", buffer);
    }
}

close(client_fd);
return 0;
}
```